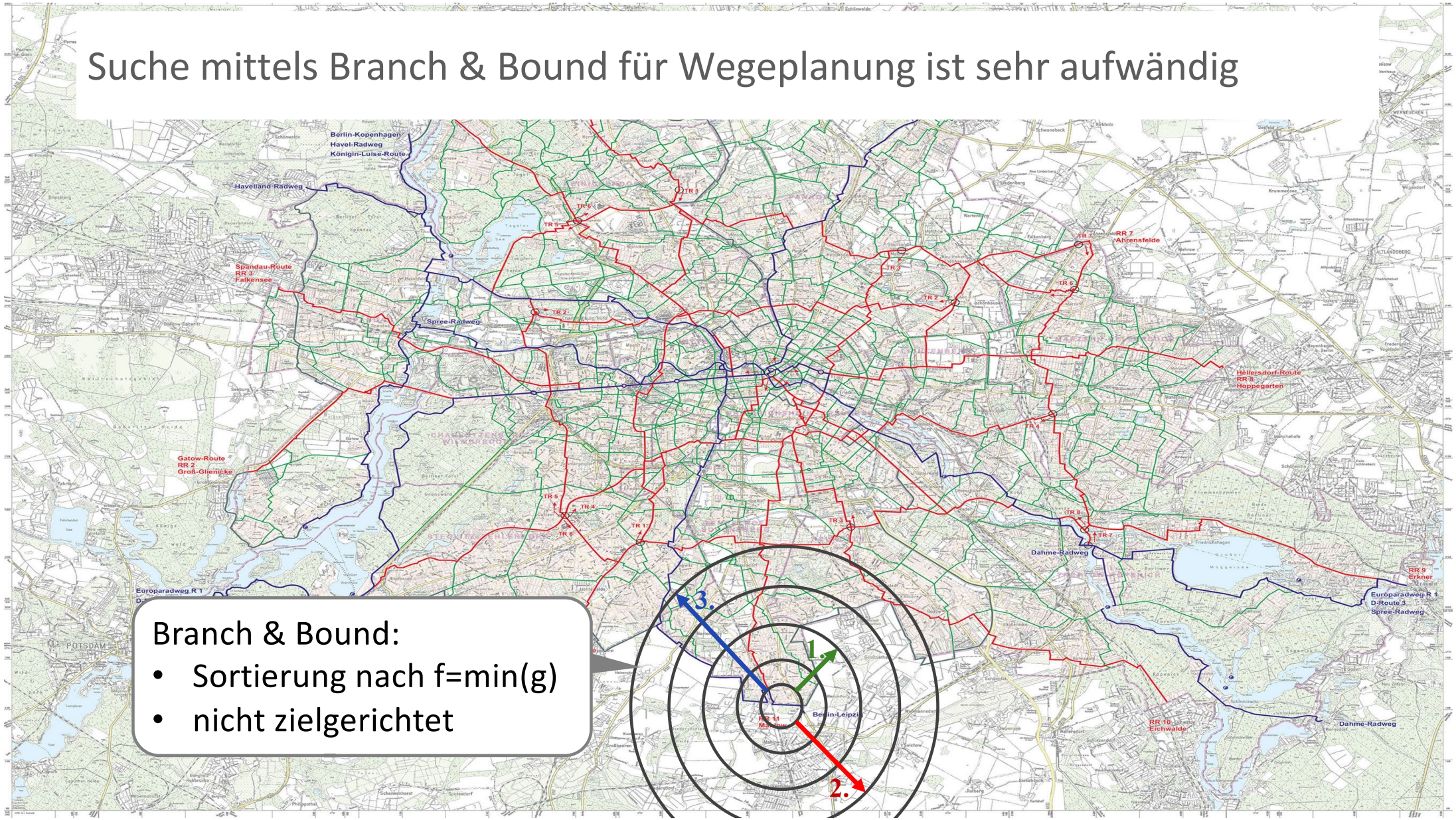
A|O|T
Agententechnologien in betrieblichen Anwendungen und der Telekommunikation

2.7 Informierte Suche

- Heuristiken
- A*

© Dr.-Ing. Stefan Fricke

Suche mittels Branch & Bound für Wegeplanung ist sehr aufwändig



Branch & Bound:

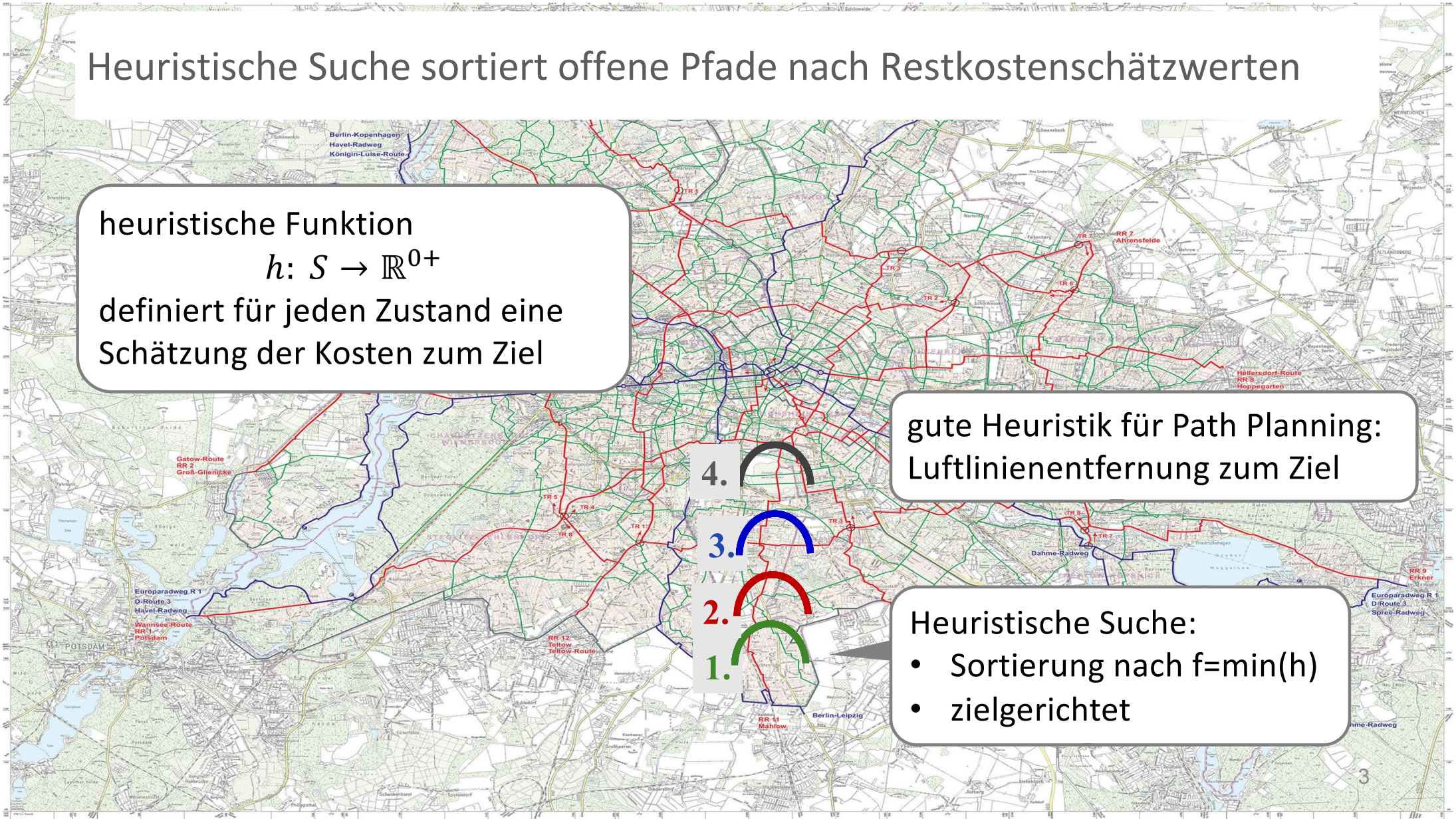
- Sortierung nach f=min(g)
- nicht zielgerichtet

Heuristische Suche sortiert offene Pfade nach Restkostenschätzwerten

heuristische Funktion

$$h\colon S \to \mathbb{R}^{0+}$$

definiert für jeden Zustand eine Schätzung der Kosten zum Ziel

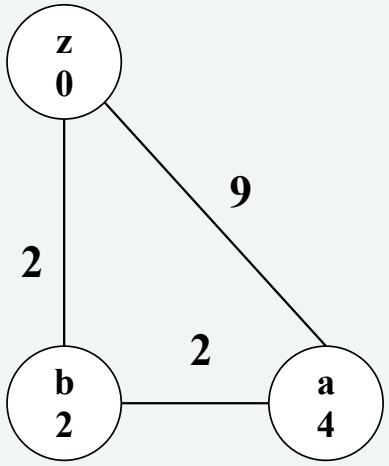


gute Heuristik für Path Planning: Luftlinienentfernung zum Ziel

Heuristische Suche:

- Sortierung nach f=min(h)
- zielgerichtet

Heuristische Suche garantiert auch mit perfekt informierter Heuristik keine Optimalität

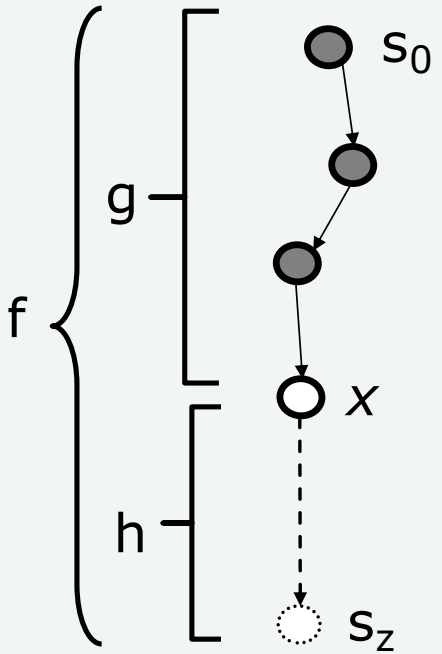


Lege Pfad (s_0) in $Open$, lege Zustand s_0 in $Generated$
Until $Open$ nicht leer
    Entferne Pfad $(s_0, \dots, s_k)$ aus $Open$ gemäß Strategie f = min(h)
    IF $zt(s_k) = true$
    THEN $return\ (s_0, \dots, s_k)$
    ELSE Für jeden Nachfolger x von $s_k \notin Generated$
        lege x in $Generated$
        lege Pfad $(s_0, \dots, s_k, x)$ in $Open$
Return Failure

A* verwendet Pfadkosten und Heuristik

Sortierung der offenen Pfade nach f = min(g + h)
- g = Pfadkosten (akkumulierte Kosten)
- h = heuristische Funktion (Vorwissen)

1. Form a one-node tree consisting of the initial state *x* as root, color root as white. Initialize DP-Table with g(*x*) = 0.
2. Until there is no white leaf
   a) Select next white leaf *x* according to min(f = g+h)
   b) If goal test on *x* succeeds, return path to current leaf
   c) Color current leaf as grey
   d) Expand current leaf, apply dynamic programming, and color the remaining child nodes white
3. Return failure.

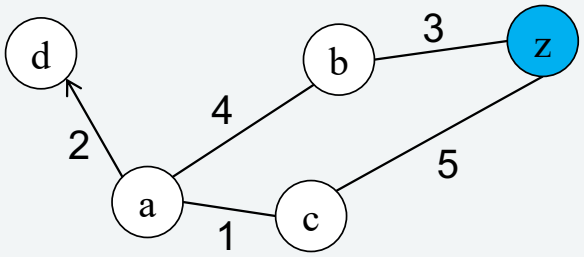


2.8 Eigenschaften von Heuristiken

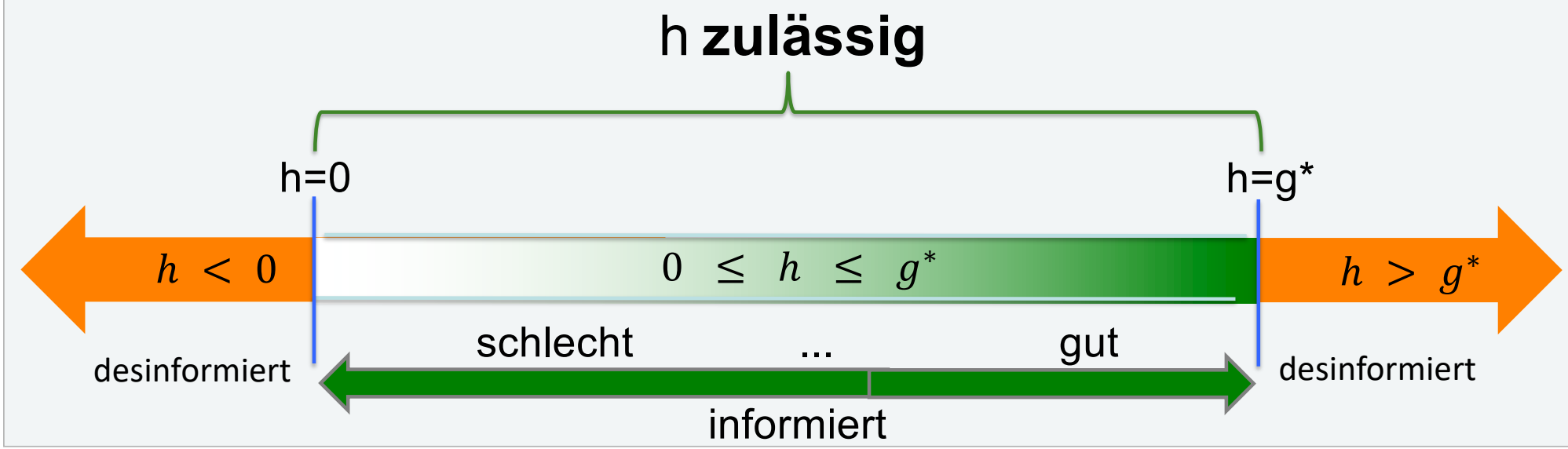
- Zulässigkeit
- Monotonie
- Dominanz

© Dr.-Ing. Stefan Fricke

Eine zulässige Heuristik überschätzt nicht die tatsächlichen Kosten

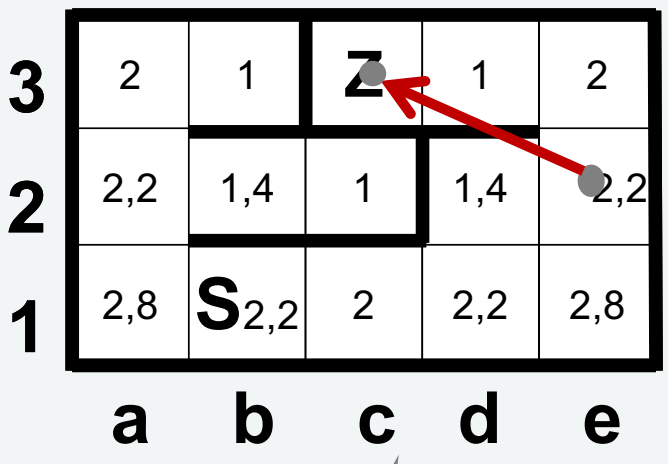


$0 \le h(a) \le 6$, $0 \le h(b) \le 3$
$0 \le h(c) \le 5$ $0 \le h(d) \le \infty$
$h(z) = 0$

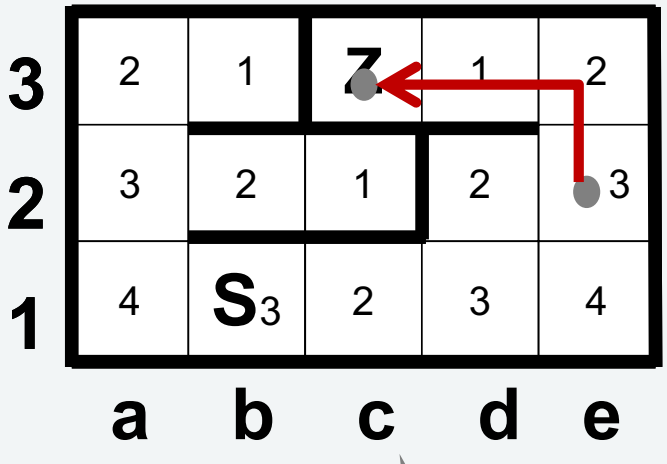


g^* : minimale Pfadkosten zum Ziel

Manhattandistanz dominiert Luftlinienentfernung



Luftlinienentfernung h_{LL} ist eine gute Heuristik



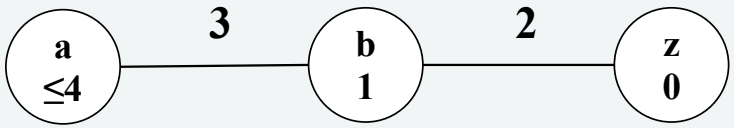
Manhattandistanz h_{MM} ist eine bessere Heuristik

Dominanz: h_{MD} zulässig und $h_{MD} \geq h_{LL}$

Monotonie ist eine noch stärkere Eigenschaft als Zulässigkeit

h ist monoton, wenn $\forall i\colon\ h(\,x_i\,) \leq\ h(\,x_{i+1}\,)\ +\ c(\,x_i, x_{i+1}\,)$

h zulässig und überschätzt außerdem auch die Aktionskosten nicht

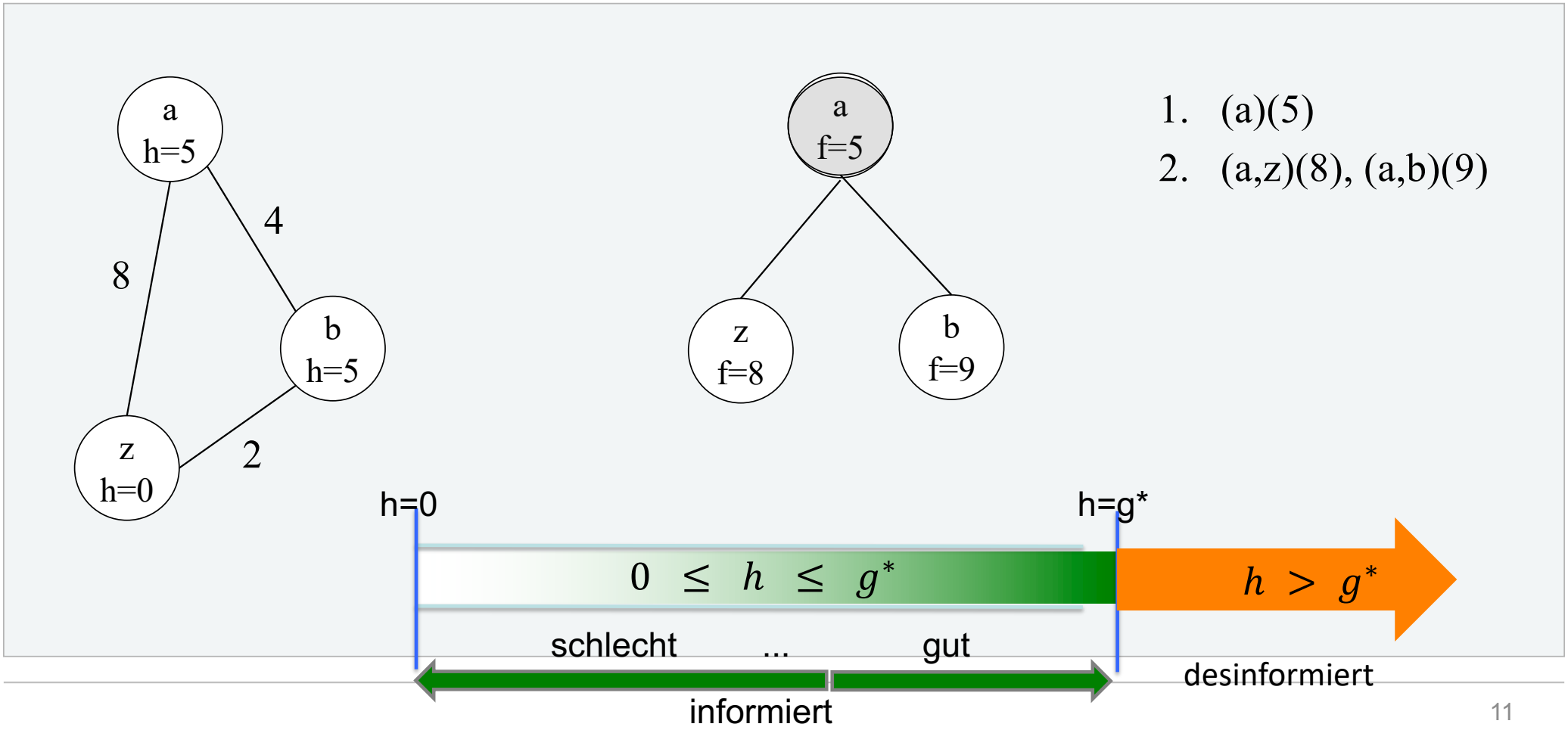


Schätzfehler (Optimismusfaktor) zum Ziel hin monoton fallend

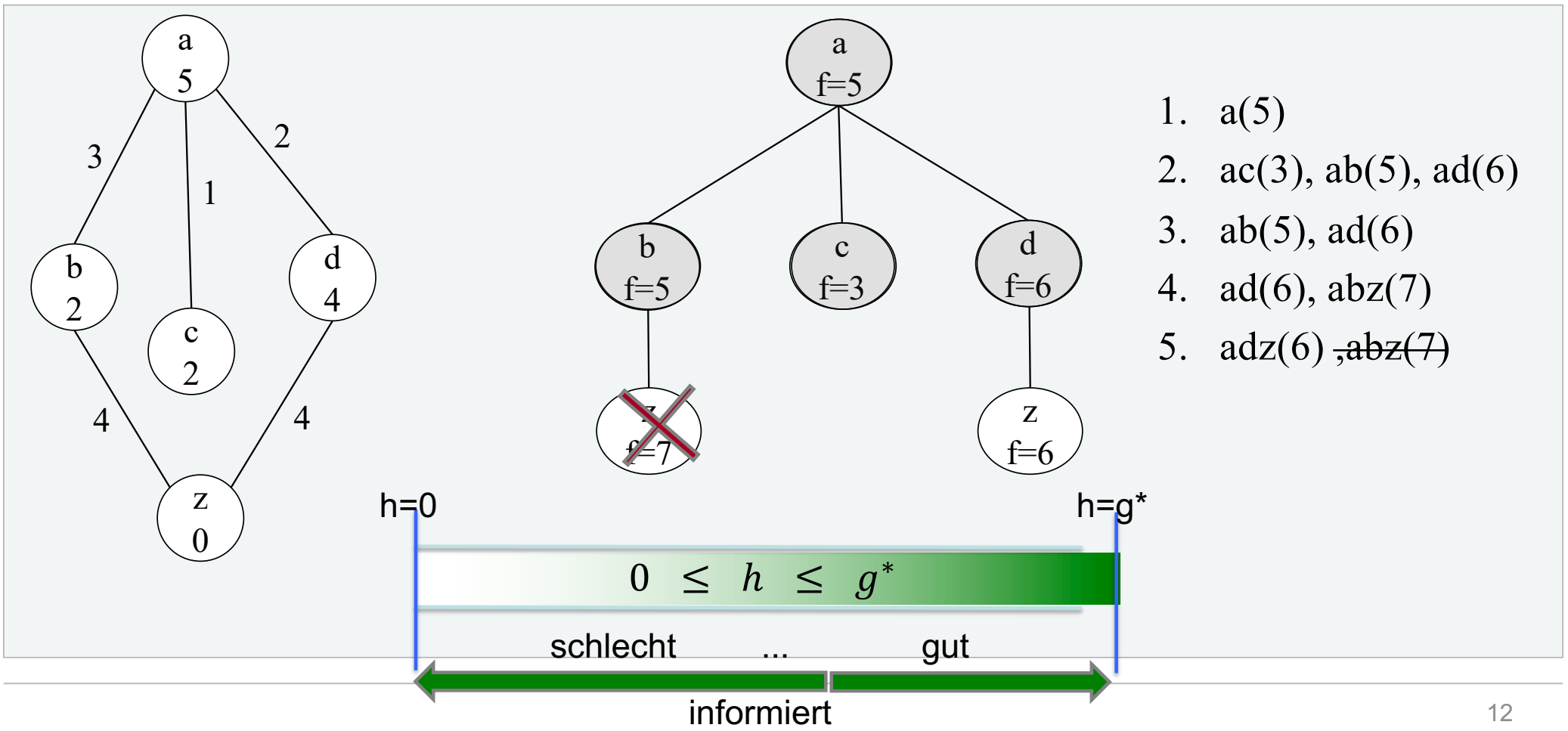
2.9 Eigenschaften von A*

© Dr.-Ing. Stefan Fricke

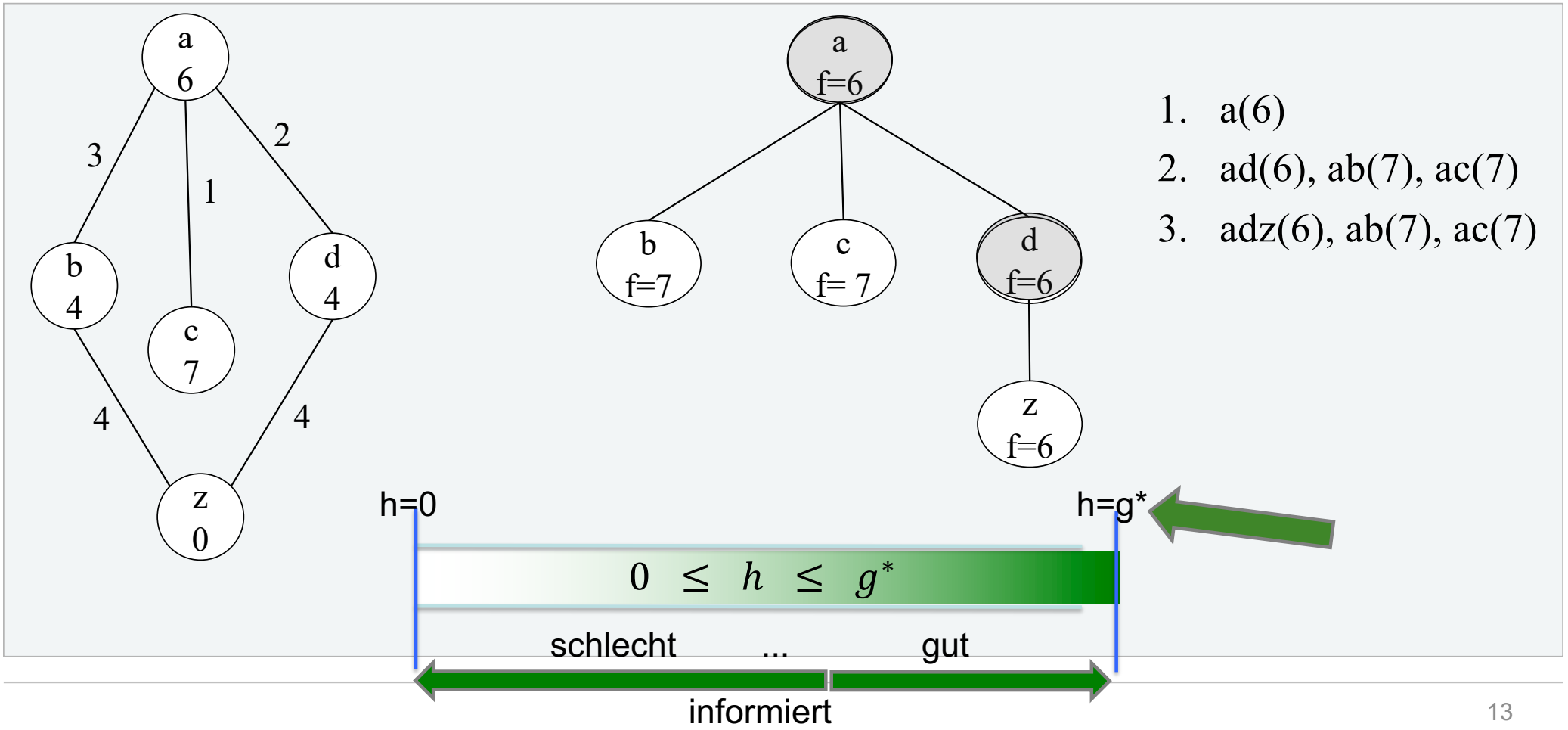
Mit unzulässiger Heuristik kann A* Optimalität nicht garantieren



Mit zulässiger Heuristik garantiert A* Optimalität

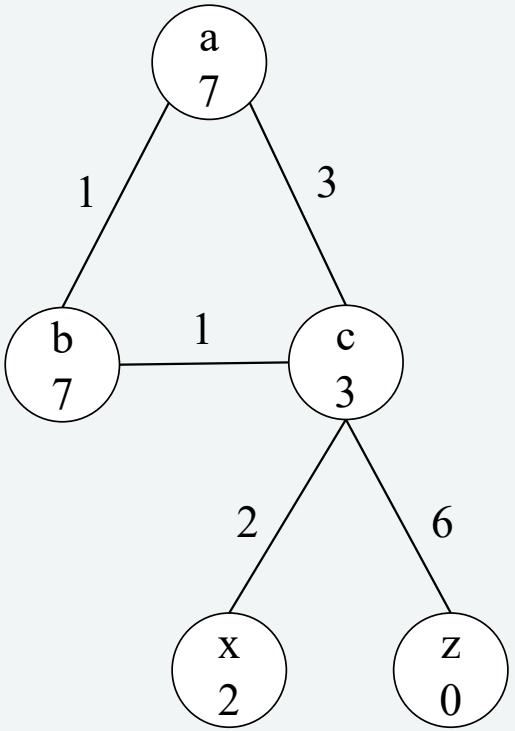


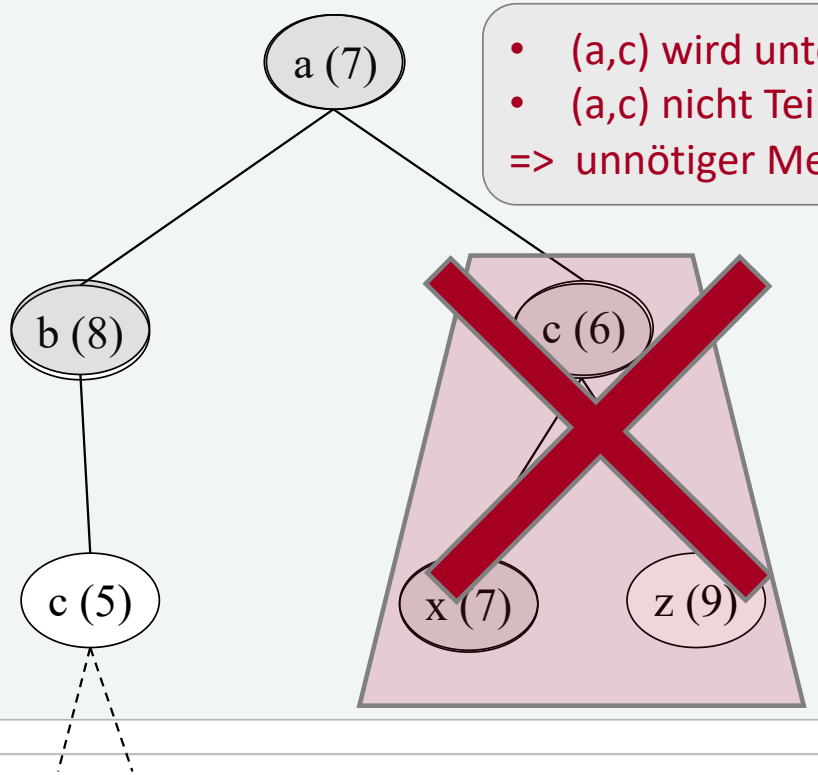
Mit optimaler Heuristik hat A* minimalen Aufwand



A* mit zulässiger aber nicht monotoner Heuristik

Monotonie: $\forall i\colon\ h(x_i) \leq h(x_{i+1}) + c(x_i, x_{i+1})$



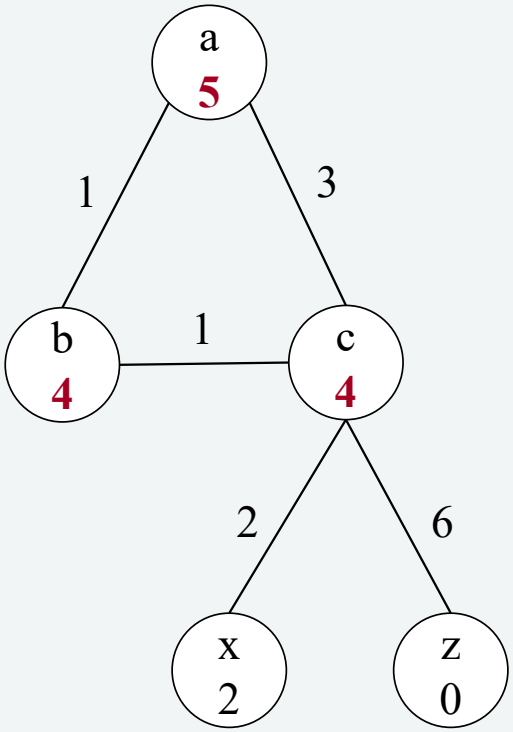


- (a,c) wird untersucht
- (a,c) nicht Teil der optimalen Lösung

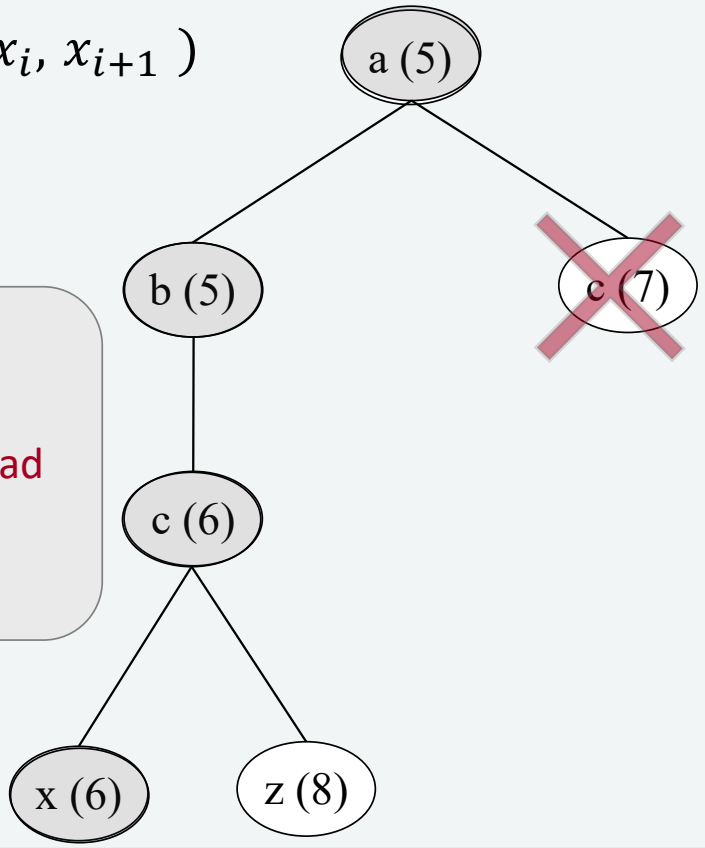
=> unnötiger Mehraufwand

Mit monotoner Heuristik besucht A* jeden Knoten auf dem kürzesten Pfad

Monotonie: $\forall i\colon\ h(x_i) \leq h(x_{i+1}) + c(x_i, x_{i+1})$



- monotone Heuristik: wachsende f-Werte
- jeder Zustand wird auf kürzestem Pfad besucht
- unnötiger Mehraufwand entfällt



A* ist ein guter Algorithmus (mit schlechten Worst-Case-Charakteristika)

Optimal effizient.
A* expandiert:

- optimistisch: Knoten mit minimalem $f(n)$
- *alle* Knoten mit $f(n) \leq g^*$
- *keine* Knoten mit $f(n) > g^*$ (dynamische Programmierung)

Mit monotoner Heuristik: Worst-Case-Laufzeit quadratisch zur Zahl der Zustände.

- Hauptproblem: Speicherverbrauch
  → Simplified memory-bounded A* [Russell, 1992]

Vollständigkeit	Ja*
Optimalität	Ja*
Zeitkomplexität	$O(b^m)$
Speicherkomplexität	$O(b^m)$

b: Verzweigungsgrad

m: maximale Suchbaumtiefe

* wenn Aktionskosten positiv und Heuristik zulässig

A|O|T
Agententechnologien in betrieblichen Anwendungen und der Telekommunikation

2.10 Lokale Suche

- Anwendungen
- erweiterte Nachbarschaft
- Simulated Annealing
- Tabusuche

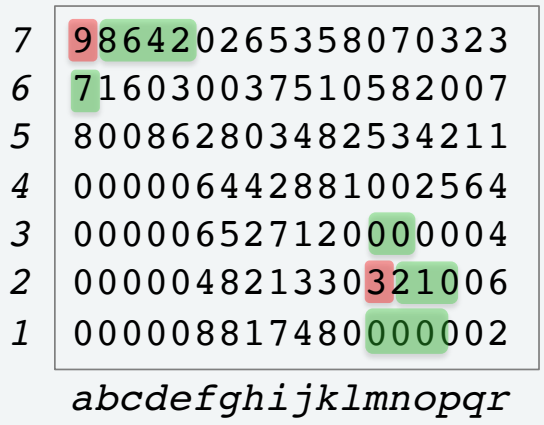
© Dr.-Ing. Stefan Fricke

Lokale Suche basiert auf iterativer Verbesserung eines Zustands

(1) determine initial candidate solution s
(2) While s is not a local optimum:
(3) choose neighbour s' of s such that h(s') < h(s)
(4) s := s'

Steepest Descend sucht *besten* Nachbarn s'*
Hill Climbing wechselt zum ersten besseren Nachbarn

Nachbarschaft ähnlich Zustandsexpandierung

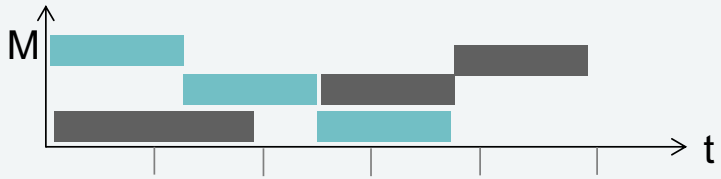


Restart ist *eine* Möglichkeit zur Flucht aus lokalem Optimum ...

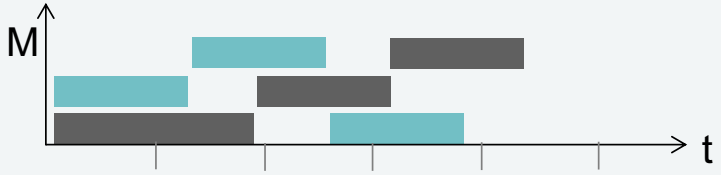
Lokale Suche ist praxisrelevant für Layoutprobleme und Scheduling

Übliche Vorgehensweise:

1. Erzeugen einer zulässigen Lösung, z.B. mittels Bestensuche (f=h)



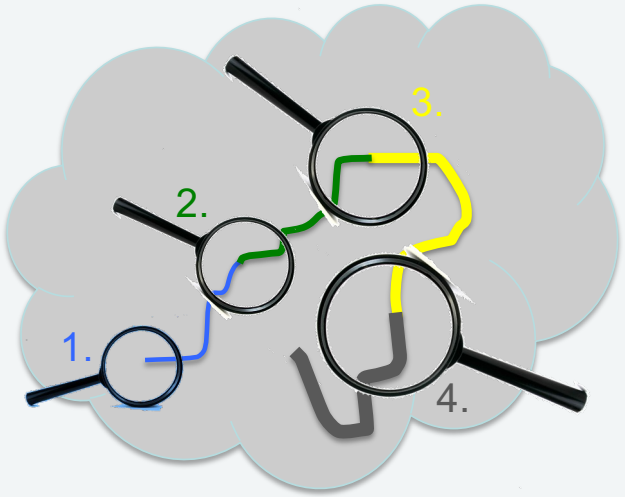
2. Optimieren mittels lokaler Suche, z.B. Verschieben & paarweises Vertauschen.



Flucht aus lokalem Optimum durch Suche in größerer Nachbarschaft

(1) determine initial candidate solution s
(2) set nhs = 1 // Size of neighbourhood
(3) While nhs ≤ maxNhs
(4) s := local search(s, nhs)
(5) nhs := nhs +1

z.B. limitierte Tiefensuche



Hoos, Stützle (2005)

Simulated Annealing ermöglicht Konvergenz durch systematisch abnehmendes Rauschen

(1) determine initial candidate solution s

(2) set initial temperature T according to annealing schedule

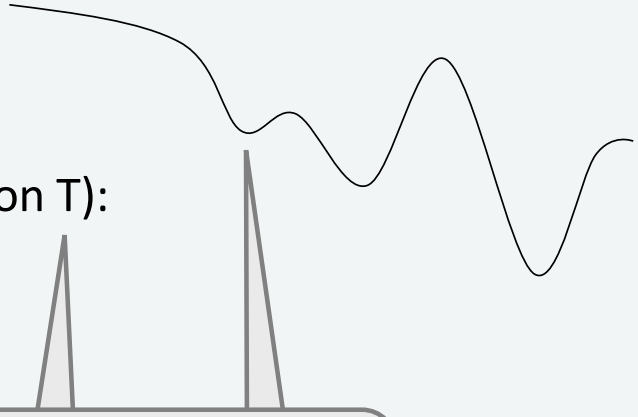
(3) While termination condition is not satisfied:

(4) probabilistically choose a neighbour s’ of s

(5) If s’ satisfies probabilistic acceptance criterion (depending on T):

(6) s := s’

(7) update T according to annealing schedule



T erlaubt Flucht aus lokalen Minima mit abnehmender Wahrscheinlichkeit

Hoos, Stützle (2005)

Tabusuche nutzt zusätzlichen Speicher zur Vermeidung lokaler Optima

(1) determine initial candidate solution s^*=s

(2) While termination criterion is not satisfied:

(3) determine set N of non-tabu neighbours of s

(4) choose a best (improving) candidate solution s' in N

(5) update tabu attributes based on s'

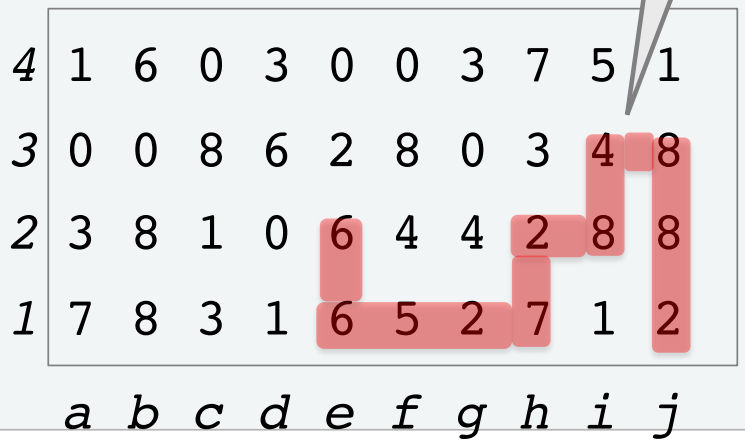
(6) if h(s´) < h(s^*) then $s^* \leftarrow$ s`

(7) s := s'

bester Nachbar nicht in Tabuliste

Tabusuche generiert Trajektorie

Speicherung der aktuellen besten Lösung notwendig



Bewertung der lokalen Suche

